Discrete-1.2.60
Local Error of Numerical Scheme

NAMASWI V – EE25BTECH11060

March 27, 2026

Outline

Question

The local error of the scheme

$$y_{n+1} = y_n + \frac{h}{12}\left(5y_{n+1} + 8y_n - y_{n-1}\right) \tag{1}$$

a) $O(h)$

b) $O(h^2)$

c) $O(h^3)$

d) $O(h^4)$ ✓

What is Local Truncation Error?

Local truncation error (LTE) measures how much a numerical scheme deviates from the exact Taylor expansion in **one step**.

Approach:

- Expand exact solution y_{n+1}, y_{n-1} using Taylor series around x_n
- Substitute into the numerical scheme
- Compare with the exact Taylor expansion of y_{n+1}
- First mismatching term $\Rightarrow$ **order of LTE**

Taylor Expansions

Expanding around x_n with step size h:

$$y_{n+1} = y_n + hy'_n + \frac{h^2}{2}y''_n + \frac{h^3}{6}y'''_n + O(h^4) \tag{2}$$

$$y_{n-1} = y_n - hy'_n + \frac{h^2}{2}y''_n - \frac{h^3}{6}y'''_n + O(h^4) \tag{3}$$

Equations (2) and (3) are substituted into the right-hand side of (1).

Substitution into the Scheme

Substituting (2) and (3) into the RHS of (1):

$$y_{n+1} = y_n + \frac{h}{12}\Big[5\Big(y_n + hy_n' + \tfrac{h^2}{2}y_n'' + \tfrac{h^3}{6}y_n'''\Big) + 8y_n - \Big(y_n - hy_n' + \tfrac{h^2}{2}y_n'' - \tfrac{h^3}{6}y_n'''\Big)\Big] \tag{4}$$

Collecting like terms inside the bracket:

$$y_{n+1} = y_n + \frac{h}{12}\left(12y_n + 6hy_n' + 2h^2y_n'' + \frac{h^3}{3}y_n''' + \cdots\right) \tag{5}$$

Simplification and LTE

Dividing through by 12:

$$y_{n+1} = y_n + hy'_n + \frac{h^2}{2}y''_n + \frac{h^3}{6}y'''_n + O(h^4) \tag{6}$$

Exact Taylor expansion:

$$y_{n+1} = y_n + hy'_n + \frac{h^2}{2}y''_n + \frac{h^3}{6}y'''_n + O(h^4) \tag{7}$$

The scheme matches all terms up to h^3. The first error appears at $O(h^4)$.

Result

Local Truncation Error $= O(h^4)$ $\Rightarrow$ **Option (d)**

Rearranging the Scheme — General Form

For a general ODE $\frac{dy}{dx} = \lambda y$, we have $y_n' = \lambda y_n$.

The scheme (1) becomes:

$$y_{n+1} = y_n + \frac{h}{12}(5\lambda y_{n+1} + 8\lambda y_n - \lambda y_{n-1})$$

$$\left(1 - \frac{5\lambda h}{12}\right) y_{n+1} = \left(1 + \frac{8\lambda h}{12}\right) y_n - \frac{\lambda h}{12} y_{n-1} \tag{8}$$

Define constants:

$$A = \frac{1 + \frac{8\lambda h}{12}}{1 - \frac{5\lambda h}{12}}, \qquad B = \frac{-\frac{\lambda h}{12}}{1 - \frac{5\lambda h}{12}} \tag{9}$$

So the recurrence is:

$$y_{n+1} = A\, y_n + B\, y_{n-1} \tag{10}$$

Z-Transform Setup

Taking the Z-transform of (10). Recall:

$$\mathcal{Z}\{y_{n+1}\} = zY(z) - zy_0, \qquad \mathcal{Z}\{y_{n-1}\} = \frac{Y(z)}{z}$$

Applying to both sides of (10):

$$zY(z) - zy_0 = A\,Y(z) + \frac{B}{z}\,Y(z) \tag{11}$$

Collecting $Y(z)$ on the left:

$$Y(z)\left(z - A - \frac{B}{z}\right) = z\,y_0 \tag{12}$$

Z-Transform Solution

Multiplying numerator and denominator by z:

$$\boxed{Y(z) = \frac{z^2 y_0}{z^2 - Az - B}} \tag{13}$$

The denominator gives the **characteristic equation**:

$$z^2 - Az - B = 0 \tag{14}$$

Solving for roots z_1, z_2:

$$z_{1,2} = \frac{A \pm \sqrt{A^2 + 4B}}{2} \tag{15}$$

The general solution is:

$$y_n = C_1 z_1^n + C_2 z_2^n \tag{16}$$

Constants C_1, C_2 are found from initial conditions y_0 and y_1.

Z-Transform: Stability Condition

The scheme is stable if all roots of $z^2 - Az - B = 0$ satisfy:

$$|z_i| \leq 1$$

Distinct real roots $\Rightarrow y_n = C_1 z_1^n + C_2 z_2^n$

Complex roots $\Rightarrow$ oscillatory form $r^n(\cos n\theta + i \sin n\theta)$

Repeated root $\Rightarrow y_n = (C_1 + C_2 n)\, z_1^n$

Physical meaning: If $|z_i| > 1$, numerical errors grow exponentially $\Rightarrow$ unstable scheme.

Example 1 — Setup: $dy/dx = y$

ODE: $\dfrac{dy}{dx} = y, \quad y(0) = 1$
Exact Solution: $y(x) = e^x$

Here $\lambda = 1$, so the scheme constants (from (9)) are:

$$A_1 = \frac{1 + \frac{8h}{12}}{1 - \frac{5h}{12}} = \frac{1 + 8h/12}{1 - 5h/12} \approx 1.1130, \qquad B_1 \approx -0.0087 \quad (h = 0.1) \tag{17}$$

Initial conditions (chosen from exact solution):

$$y_0 = 1, \qquad y_1 = e^h$$

Example 1 — Python Code (FD Scheme)

```
import math, matplotlib.pyplot as plt

h = 0.1;  N = 20

x     = [i*h for i in range(N+1)]
y_ex  = [math.exp(xi) for xi in x]          # exact: e^x

# FD scheme constants (lambda = 1)
A = (1 + 8*h/12) / (1 - 5*h/12)
B = (-h/12)      / (1 - 5*h/12)

# initial conditions from exact solution
y_fd = [1.0, math.exp(h)] + [0.0]*(N-1)

for n in range(1, N):
    y_fd[n+1] = A * y_fd[n] + B * y_fd[n-1]

error = [abs(y_ex[i] - y_fd[i]) for i in range(N+1)]
```

Example 1 — Python Code (RK4)

```
def f(x, y): return y            # dy/dx = y

y_rk = [1.0] + [0.0]*N          # y(0) = 1

for n in range(N):
    xi, yi = x[n], y_rk[n]
    k1 = f(xi,        yi)
    k2 = f(xi + h/2, yi + h/2 * k1)
    k3 = f(xi + h/2, yi + h/2 * k2)
    k4 = f(xi + h,   yi + h   * k3)
    y_rk[n+1] = yi + (h/6) * (k1 + 2*k2 + 2*k3 + k4)

err_rk = [abs(y_ex[i] - y_rk[i]) for i in range(N+1)]
```

RK4 idea: Approximate slope using weighted average of four slope estimates k_1, k_2, k_3, k_4 over the interval $[x_n,\ x_n + h]$.

Example 1 — Plot Code: Exact Solution

```
# Plot 1: Exact solution only
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex, 'b-o', markersize=5, linewidth=2,
         label=r'Exact: $e^x$')
plt.title(r'Exact Solution: $dy/dx = y$')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex1_exact.pdf')
plt.show()
```

Example 1 — Exact Solution

ex1_exact.pdf

Example 1 — Plot Code: FD Scheme vs Exact

```
# Plot 2: FD Scheme vs Exact
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex, 'b-o',  markersize=5, linewidth=2,
         label=r'Exact: $e^x$')
plt.plot(x, y_fd, 'r--s', markersize=5, linewidth=1.8,
         label='Finite Difference Scheme')
plt.title('Example 1: Exact vs Finite Difference')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex1_fd.pdf')
plt.show()
```

Example 1 — FD Scheme vs Exact

ex1_fd.pdf

Example 1 — Plot Code: RK4 vs Exact

```
# Plot 3: RK4 vs Exact
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex,  'b-o',  markersize=5, linewidth=2,
         label=r'Exact: $e^x$')
plt.plot(x, y_rk,  'g--^', markersize=5, linewidth=1.8,
         label='RK4')
plt.title('Example 1: Exact vs RK4')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex1_rk4.pdf')
plt.show()
```

Example 1 — RK4 vs Exact

ex1_rk4.pdf

Example 1 — Plot Code: All Methods Combined

```
# Plot 4: All methods combined
plt.figure(figsize=(7, 4.5))
plt.plot(x, y_ex,  'b-o',   markersize=5, linewidth=2,
         label=r'Exact: $e^x$')
plt.plot(x, y_fd,  'r--s',  markersize=5, linewidth=1.8,
         label='Finite Difference Scheme')
plt.plot(x, y_rk,  'g-.^',  markersize=5, linewidth=1.8,
         label='RK4')
plt.title(r'Example 1: All Methods ($dy/dx = y$)')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex1_combined.pdf')
plt.show()
```

Example 1 — All Methods Combined

ex1_combined.pdf

Example 1 — Plot Code: FD Scheme Error

```
# error list (computed earlier)
error_fd = [abs(y_ex[i] - y_fd[i]) for i in range(N+1)]

# Plot 5: FD Scheme Error
plt.figure(figsize=(6, 3.5))
plt.semilogy(x, error_fd, 'r-o', markersize=5, linewidth=1.8)
plt.title(r'FD Scheme Error (Ex 1): $|e^x - y_{FD}|$')
plt.xlabel('x')
plt.ylabel('Absolute Error (log scale)')
plt.grid(True, which='both', alpha=0.4)
plt.tight_layout()
plt.savefig('ex1_err_fd.pdf')
plt.show()
```

Example 1 — Error: FD Scheme

ex1_err_fd.pdf

Example 1 — Plot Code: RK4 Error

```
# error list (computed earlier)
error_rk = [abs(y_ex[i] - y_rk[i]) for i in range(N+1)]

# Plot 6: RK4 Error
plt.figure(figsize=(6, 3.5))
plt.semilogy(x, error_rk, 'g-^', markersize=5, linewidth=1.8)
plt.title(r'RK4 Error (Ex 1): $|e^x - y_{RK4}|$')
plt.xlabel('x')
plt.ylabel('Absolute Error (log scale)')
plt.grid(True, which='both', alpha=0.4)
plt.tight_layout()
plt.savefig('ex1_err_rk4.pdf')
plt.show()
```

Example 1 — Error: RK4

ex1_err_rk4.pdf

Example 2 — Setup: $dy/dx = -2y$

ODE: $\dfrac{dy}{dx} = -2y, \quad y(0) = 1$
Exact Solution: $y(x) = e^{-2x}$

Here $\lambda = -2$. Substituting into (9):

$$A_2 = \frac{1 + \frac{8(-2)h}{12}}{1 - \frac{5(-2)h}{12}} = \frac{1 - \frac{16h}{12}}{1 + \frac{10h}{12}} \approx 0.8000, \qquad B_2 \approx 0.0154 \quad (h = 0.1) \quad (18)$$

Initial conditions:

$$y_0 = 1, \qquad y_1 = e^{-2h}$$

Note: A_2, B_2 are **different** from Example 1 because λ changed.

Example 2 — Python Code (FD Scheme)

```
import math, matplotlib.pyplot as plt

h = 0.1;  N = 20

x      = [i*h for i in range(N+1)]
y_ex   = [math.exp(-2*xi) for xi in x]        # exact: e^{-2x}

# FD scheme constants (lambda = -2)
A = (1 - 16*h/12) / (1 + 10*h/12)
B = ( 2*h/12)     / (1 + 10*h/12)

# initial conditions
y_fd = [1.0, math.exp(-2*h)] + [0.0]*(N-1)

for n in range(1, N):
    y_fd[n+1] = A * y_fd[n] + B * y_fd[n-1]

error = [abs(y_ex[i] - y_fd[i]) for i in range(N+1)]
```

Example 2 — Python Code (RK4)

```
def f(x, y): return -2 * y     # dy/dx = -2y

y_rk = [1.0] + [0.0]*N         # y(0) = 1

for n in range(N):
    xi, yi = x[n], y_rk[n]
    k1 = f(xi,        yi)
    k2 = f(xi + h/2, yi + h/2 * k1)
    k3 = f(xi + h/2, yi + h/2 * k2)
    k4 = f(xi + h,   yi + h   * k3)
    y_rk[n+1] = yi + (h/6) * (k1 + 2*k2 + 2*k3 + k4)

err_rk = [abs(y_ex[i] - y_rk[i]) for i in range(N+1)]
```

Example 2 — Plot Code: Exact Solution

```
# Plot 1: Exact solution only
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex, 'b-o', markersize=5, linewidth=2,
         label=r'Exact: $e^{-2x}$')
plt.title(r'Exact Solution: $dy/dx = -2y$')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex2_exact.pdf')
plt.show()
```

Example 2 — Exact Solution

ex2_exact.pdf

Example 2 — Plot Code: FD Scheme vs Exact

```
# Plot 2: FD Scheme vs Exact
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex, 'b-o',  markersize=5, linewidth=2,
         label=r'Exact: $e^{-2x}$')
plt.plot(x, y_fd, 'r--s', markersize=5, linewidth=1.8,
         label='Finite Difference Scheme')
plt.title('Example 2: Exact vs Finite Difference')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex2_fd.pdf')
plt.show()
```

Example 2 — FD Scheme vs Exact

ex2_fd.pdf

Example 2 — Plot Code: RK4 vs Exact

```
# Plot 3: RK4 vs Exact
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex,  'b-o',  markersize=5, linewidth=2,
         label=r'Exact: $e^{-2x}$')
plt.plot(x, y_rk,  'g--^', markersize=5, linewidth=1.8,
         label='RK4')
plt.title('Example 2: Exact vs RK4')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex2_rk4.pdf')
plt.show()
```

Example 2 — RK4 vs Exact

ex2_rk4.pdf

Example 2 — Plot Code: All Methods Combined

```
# Plot 4: All methods combined
plt.figure(figsize=(7, 4.5))
plt.plot(x, y_ex,  'b-o',   markersize=5, linewidth=2,
         label=r'Exact: $e^{-2x}$')
plt.plot(x, y_fd,  'r--s',  markersize=5, linewidth=1.8,
         label='Finite Difference Scheme')
plt.plot(x, y_rk,  'g-.^',  markersize=5, linewidth=1.8,
         label='RK4')
plt.title(r'Example 2: All Methods ($dy/dx = -2y$)')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('ex2_combined.pdf')
plt.show()
```

Example 2 — All Methods Combined

ex2_combined.pdf

Example 2 — Plot Code: FD Scheme Error

```
# error list (computed earlier)
error_fd = [abs(y_ex[i] - y_fd[i]) for i in range(N+1)]

# Plot 5: FD Scheme Error
plt.figure(figsize=(6, 3.5))
plt.semilogy(x, error_fd, 'r-o', markersize=5, linewidth=1.8)
plt.title(r'FD Scheme Error (Ex 2): $|e^{-2x} - y_{FD}|$')
plt.xlabel('x')
plt.ylabel('Absolute Error (log scale)')
plt.grid(True, which='both', alpha=0.4)
plt.tight_layout()
plt.savefig('ex2_err_fd.pdf')
plt.show()
```

Example 2 — Error: FD Scheme

ex2_err_fd.pdf

Example 2 — Plot Code: RK4 Error

```
# error list (computed earlier)
error_rk = [abs(y_ex[i] - y_rk[i]) for i in range(N+1)]

# Plot 6: RK4 Error
plt.figure(figsize=(6, 3.5))
plt.semilogy(x, error_rk, 'g-^', markersize=5, linewidth=1.8)
plt.title(r'RK4 Error (Ex 2): $|e^{-2x} - y_{RK4}|$')
plt.xlabel('x')
plt.ylabel('Absolute Error (log scale)')
plt.grid(True, which='both', alpha=0.4)
plt.tight_layout()
plt.savefig('ex2_err_rk4.pdf')
plt.show()
```

Example 2 — Error: RK4

ex2_err_rk4.pdf

RK4 Method — What is it?

The **4th-order Runge-Kutta (RK4)** method solves $\frac{dy}{dx} = f(x, y)$ by computing four slope estimates per step:

$$k_1 = f(x_n,\ y_n) \tag{19}$$

$$k_2 = f\left(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1\right) \tag{20}$$

$$k_3 = f\left(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_2\right) \tag{21}$$

$$k_4 = f(x_n + h,\ y_n + h\,k_3) \tag{22}$$

$$\boxed{y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)} \tag{23}$$

RK4 — Why $O(h^4)$?

The Taylor expansion of the exact solution is:

$$y(x+h) = y + hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \frac{h^4}{24}y^{(4)} + \cdots$$

RK4 matches **all four terms** exactly through clever weighted averaging:

Slope	Evaluation point
k_1	Left endpoint: x_n
k_2	Midpoint using k_1
k_3	Midpoint using k_2 (corrected)
k_4	Right endpoint using k_3

Local truncation error of RK4:

$$\text{LTE}_{RK4} = O(h^5) \quad \Rightarrow \quad \text{Global error} = O(h^4) \tag{24}$$

Method Comparison

Property	**Given Scheme**	**RK4**
Type	Multistep	Single-step
Values needed	y_n, y_{n-1}	y_n only
LTE	$O(h^4)$	$O(h^5)$
Global error	$O(h^3)$	$O(h^4)$
Startup	Needs y_1 separately	Only needs y_0
ODE-dependency	A, B depend on λ	Only $f(x, y)$
Implicit	Yes (for implicit ODEs)	No

Both methods are high-order, but RK4 has lower error per step.

Summary and Conclusion

The scheme $y_{n+1} = y_n + \frac{h}{12}(5y_{n+1} + 8y_n - y_{n-1})$ matches the exact Taylor expansion up to and including the h^3 term

The first deviation appears at $O(h^4) \Rightarrow$ **LTE =** $O(h^4)$

Z-transform converts the recurrence to an algebraic equation, giving the general solution via the characteristic equation $z^2 - Az - B = 0$

Numerical tests on both $dy/dx = y$ and $dy/dx = -2y$ confirm: errors are small and consistent with $O(h^4)$

RK4, also $O(h^4)$ globally, gives comparable accuracy

Final Answer

$\boxed{\text{Local Truncation Error} = O(h^4)}$ **Option (d)**

Plotting y_n Directly from the Recurrence

Given:

$$y_{n+1} = A\,y_n + B\,y_{n-1}, \qquad A = \frac{1+\frac{8h}{12}}{1-\frac{5h}{12}}, \quad B = \frac{-\frac{h}{12}}{1-\frac{5h}{12}}$$

Chosen initial conditions and step size:

$$y_0 = 1, \qquad y_1 = 1.1, \qquad h = 0.1$$

$y_1 = 1.1$ is **not** the exact $e^{0.1} \approx 1.10517$ — this deliberate small difference lets us observe how an initial error propagates through the recurrence.

We then:

Plot the computed y_n sequence

Compare with exact e^{x_n}

Show absolute error $|e^{x_n} - y_n|$

Python Code: Computing y_n

```python
import math
import matplotlib.pyplot as plt

h = 0.1;    N = 20

# recurrence constants (dy/dx = y, lambda = 1)
A = (1 + 8*h/12) / (1 - 5*h/12)
B = (-h/12)      / (1 - 5*h/12)

# chosen initial conditions
y0, y1 = 1.0, 1.1           # y1=1.1, not exact e^h

x    = [i * h for i in range(N + 1)]
y_fd = [y0, y1] + [0.0] * (N - 1)

for n in range(1, N):
    y_fd[n + 1] = A * y_fd[n] + B * y_fd[n - 1]

y_ex  = [math.exp(xi) for xi in x]
error = [abs(y_ex[i] - y_fd[i]) for i in range(N + 1)]
```

Python Code: Plot y_n Sequence

```
# Plot 1: yn sequence only
plt.figure(figsize=(6, 4))
plt.plot(x, y_fd, 'r-o', markersize=5, linewidth=2,
         label=r'$y_n$ (Recurrence)')
plt.title(r'$y_n$ from Recurrence: $y_0=1,\ y_1=1.1,\ h=0.1$')
plt.xlabel(r'$x_n = nh$')
plt.ylabel(r'$y_n$')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```

y_n Sequence from Recurrence

yn_sequence.pdf

Python Code: y_n vs Exact e^x

```
# Plot 2: yn vs exact
plt.figure(figsize=(6, 4))
plt.plot(x, y_ex, 'b-o',  markersize=5, linewidth=2,
         label=r'Exact: $e^x$')
plt.plot(x, y_fd, 'r--s', markersize=5, linewidth=1.8,
         label=r'$y_n$ (Recurrence, $y_1=1.1$)')
plt.title(r'$y_n$ vs Exact $e^x$  ($y_0=1,\ y_1=1.1,\ h=0.1$)')
plt.xlabel(r'$x_n = nh$')
plt.ylabel(r'$y_n$')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```

y_n vs Exact e^x

yn_vs_exact.pdf

Python Code: Error Plot

```
# Plot 3: absolute error
plt.figure(figsize=(6, 3.5))
plt.semilogy(x, error, 'k-o', markersize=5, linewidth=1.8)
plt.title(r'Error $|e^{x_n} - y_n|$  ($y_0=1,\ y_1=1.1,\ h=0.1$)')
plt.xlabel(r'$x_n = nh$')
plt.ylabel('Absolute Error (log scale)')
plt.grid(True, which='both', alpha=0.4)
plt.tight_layout()
plt.show()
```

Error: $\left|e^{x_n} - y_n\right|$

yn_error.pdf